Προηγμένα Θέματα Τεχνολογίας Λογισμικού

Τελική Εργασία: Intelligent Logistics API
Γιώργος Στυλιανόπουλος

GitHub Repository link: Intelligent Logistics API

Τεχνικές απαιτήσεις

- **Γλώσσα:** Java 21 (JDK 21)
- **Framework:** 3.2.1
- **DataBase:** PostgreSQL
- **Build Tool:** Maven

Δημιουργία και εκκίνηση απαραιτήτων λειτουργιών

Η βάση δεδομένων της εργασίας είναι PostgreSQL και είναι εγκαταστημένη σε docker σε localhost. Η πόρτα της βάσης είναι η **5432**. Το Docker Compose file βρίσκεται στο root directory του project υπό την ονομασία docker-compose.yml. Για την δημιουργία του docker και της βάσης δεδομένων, εκτελείται η εντολή **docker-compose up -d postgres-db**. Η σύνδεση της βάσης με το project επιτυγχάνεται μέσω του αρχείου application.properties. Στο directory **src/main/resources** βρίσκεται το **schema.sql** που δημιουργεί τους πίνακες της βάσης δεδομένων καθώς και το **data.sql** που ενημερώνει τους πίνακες με δεδομένα. Στη συνέχεια, η main class του project εκτελεί ένα «Internal Warm-up» με σκοπό την αντιμετώπιση του «Lazy Loading» και του «Cold Start» των συνδέσεων στέλνοντας dummy δεδομένα που αντιμετωπίζονται αναλόγως από το πρόγραμμα. Περισσότερη ανάλυση θα δωθεί στην ενότητα . Για την ενεργοποίηση της ικανότητας ασύγχρονης εκτέλεσης του Spring Boot απαραίτητη είναι η προσθήκη του Annotation **@EnableAsync** στη main class. Επίσης, στο root directory βρίσκεται και το Postman Collection που περιέχει όλα τα queries στα αντίστοιχα endpoints καθώς και με ένα query με λανθασμένες παραμέτρους για κάθε endpoint.

Δομή των κλάσεων

Διαχείρηση Παραγγελιών

Σε αυτή τη την ενότητα θα αναλυθεί η **διαχείρηση παραγγελιών.** Το προγραμματιστικό πρότυπο που ακολουθήθηκε είναι το «Strategy». Όταν ο client στείλει POST στο URI: http://localhost:8080/orders, με **body:** {"customerName":"Ziggy STANDARD", "weight": 7, "destination": "Mars", "shippingType": "STANDARD"} το ShippingController στέλνει αμέσως respond: 200 OK και "PENDING" σε λιγότερο απο 200ms ενώ ένα δεύτερο thread υπολογίζει το κόστος παραγγελίας και το εκτυπώνει. Η μέθοδος processOrderInBackground εκτελείται σε ξεχωριστό thread pool (μέσω της @Async), διασφαλίζοντας ότι οι βαριές λειτουργίες (π.χ. υπολογισμοί δρομολόγησης, εγγραφή στη βάση) δεν «παγώνουν» το main Thread. Για τη χρήση multi-threading χωρίς να γεμίσει η μνήμη από threads για κάθε αίτημα η κλάση «AsyncConfig» δημιουργεί έναν αριθμό από κατελάχιστον 5 ενεργά threads με μέγιστα 10 ενεργά, με αναμονή έως και 25 εργασίες σε ουρά αναμονής εφόσον τα υπόλοιπα threads είναι απασχολημένα.
Παράλληλα, χρησιμοποιώντας το annotation @Component σε κάθε υλοποίηση του ShippingStrategy Interface, το Spring σαρώνει το classpath κατά την εκκίνηση. Η κλάση ShippingContext δέχεται στον constructor της μία λίστα απο ShippingStrategy, αυτό βοηθάει το Framework να εντοπίσει όλες τις διαθέσιμες στρατηγικές χρέωσης και να τις τοποθετεί στη λίστα αυτόματα.
Στις πρώτες εκτελέσεις του client query παρατηρήθηκε πως πάντοτε τα πρώτα respond επιστρέφονταν πάντα σε χρόνο μεγαλύτερο των 200ms. Το speedup που παρατηρείται μεταξύ της πρώτης εκτέλεσης του κώδικα του API endpoint και των επόμενων κλήσεων εμφανώς οφείλεται σε κάποιο cachng. Αυτό μπορεί να συμβαίνει είτε λόγω του JIT ByteCode optimization ώστε να χρησιμοποιούνται τα επόμενα API requests ως precompiled κώδικας είτε επιτάχυνσης του TCP connection μετά το αρχικό handshake. Για να ενεργοποιηθεί η διαδικασία

του speedup με την εκκίνηση της εφαρμογής και όχι έπειτα από τη πρώτη κλήση, με την εκκίνηση του project αμέσως εκτελείται ένα query στο συγκεκριμένο endpoint με dummy δεδομένα επιτυγχάνοντας το caching του συγκεκριμένου endpoint. Το query αυτό γίνεται consumed από τη διαχειριστική κλάση αυτών των query, δηλαδή την OrderAsyncService η οποία αναγνωρίζει το dummy body του (ή αλλιως: "WARMUP_BOT") και το αγνοεί και έτσι τα επόμενα queries επιτυγχάνονται ακαριαία.

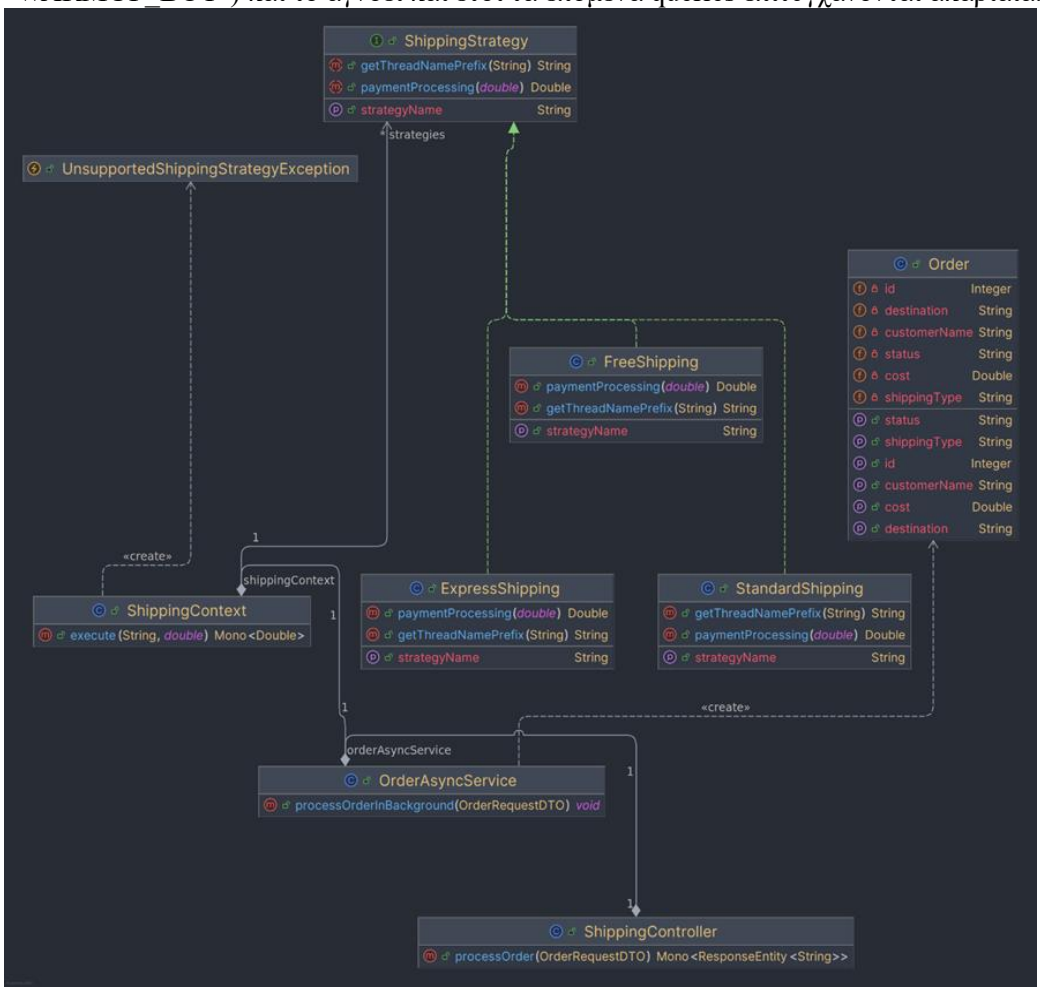

Figure 1 Strategy Pattern UML

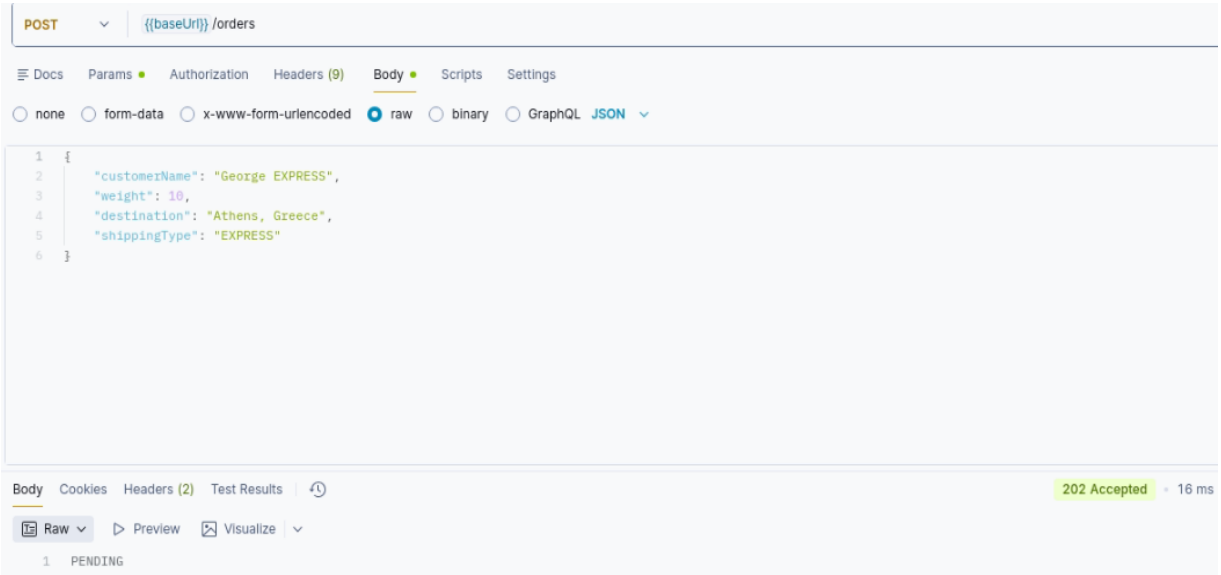

Figure 2 Παράδειγμα με express χρέσωη

Στρατηγική κοστολόγησης

Σε αυτή την ενότητα θα αναλυθεί η **στρατηγική κοστολόγησης.** Το προγραμματιστικό πρότυπο που ακολουθήθηκε είναι το «Abstract Factory». Όταν ο client στείλει GET στο URI http://localhost:8080/vehicles/init?type=Drone, το Controller «VehicleController» καλεί τη κλάση LogisticsVehicleFactory η οποία βρίσκει τη κατάλληλη υλοποίηση του VehicleFactory βάση του body που έστειλε ο χρήστης. Ύστερα με τη χρήση των αφαιρετικών κλάσεων, το αντικείμενο που έχει δημιουργηθεί ως Drone για παράδειγμα επιτρέπει στο VehicleRepository να το αποθηκεύσει στη βάση δεδομένων χρησιμοποιώντας το Single Table Inheritance, όπου η διάκριση του τύπου γίνεται αυτόματα μέσω του @DiscriminatorColumn. Στη συνέχεια η διαδικασία ολοκληρώνεται με την επιστροφή της πληροφορίας στον χρήστη, το service ανακτά τη λίστα των οχημάτων (μέσω του findByType) για να διασφαλίσει ότι επιστρέφεται η τρέχουσα κατάσταση της βάσης. Όταν η λίστα με τα Vehicle επιστρέψει στον VehicleController, ξεκινά η διαδικασία του mapping. Κάθε αντικείμενο Vehicle (πολυμορφικά) μετατρέπεται σε ένα VehicleResponseDTO ώστε να προστατεύσει εσωτερικά δεδομένα της βάσης, όπως το ID, στέλνοντας στον χρήστη μόνο αυτό που χρειάζεται σε μορφή JSON.
Τέλος η χρήση του ResponseEntity επιτρέπει τον πλήρη έλεγχο των HTTP headers και των status codes, παρέχοντας μια επαγγελματική και τυποποιημένη απάντηση (RESTful standards)

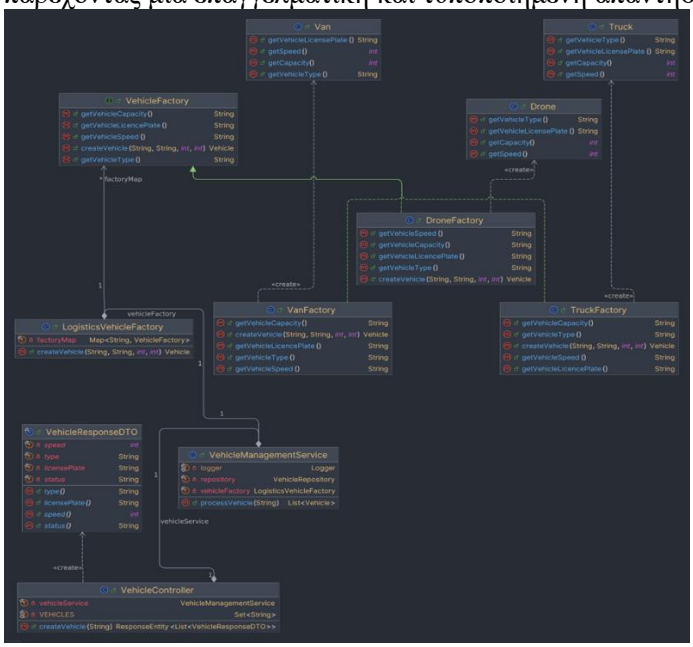

Figure 2 Abstract Factory pattern UML

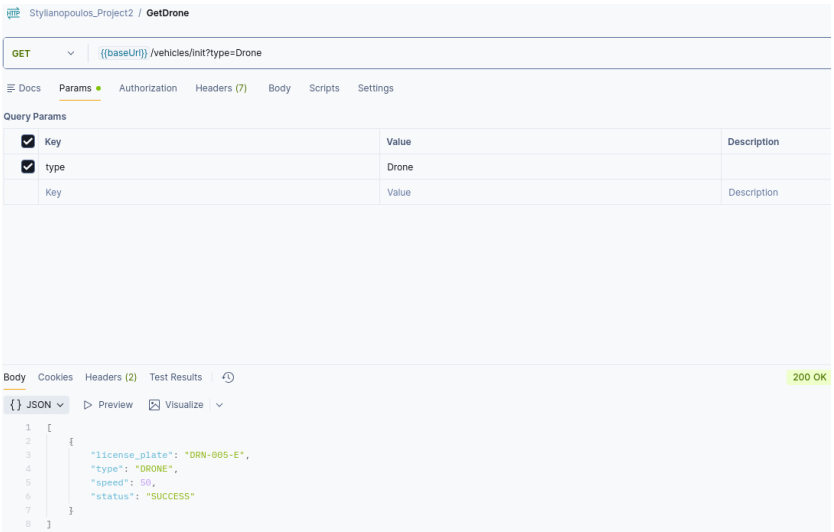

Figure 3 Παράδειγμα με type=Drone

Checklist Αυτοαξιολόγισης

Ζητούμενο (Requirement)	Κατάσταση (ΝΑΙ / ΟΧΙ / ΜΕΡΙΚΩΣ)	Σχόλια / Παρατηρήσεις
1. Basic Setup: Η εφαρμογή τρέχει (Spring Boot) και συνδέεται με τη Βάση.	ΝΑΙ	Η βάση βρίσκεται σε docker
2. Async Processing: Η παραγγελία απαντάει άμεσα, η επεξεργασία γίνεται σε background thread.	ΝΑΙ	Η κλάση και η μέθοδος στην οποία γίνεται το background task είναι τα αρχεία OrderAsyncService και processOrderInBackground() αντίστοιχα που βρίσκονται στο strategy package
3. Strategy Pattern: Υλοποίηση διαφορετικών στρατηγικών χρέωσης (Standard, Express κ.λπ.) μέσω Interface.	ΝΑΙ	Οι υλοποιήσεις του Strategy Pattern βρίσκκονται στα config, controller.api, service, service.strategy packages
4. Factory Pattern: Δημιουργία αντικειμένων Vehicle μέσω Factory κλάσης.	ΝΑΙ	Οι υλοποιήσεις του Abstract Factory pattern βρίσκονται στα controller.api, model, impl, repository, service.factory packages
5. Stream API: Υπολογισμός Analytics (Group By, Max, Filter) χωρίς SQL queries.	ΝΑΙ	Οι υλοποιήσεις βρίσκονται στη κλάση AnalyticsController
6. Clean Code: Χρήση DTOs (Request/Response objects) και όχι Entities στον Controller.	ΝΑΙ	Οι controllers χρησιμοποιήθηκαν μόνο ως endpoints και καλούν τη κλάση που αντιστοιχεί σε κάθε endpoint.